

Capítulo 1

El Lenguaje de Dominio Específico

1.1 Visión general

La generación de libros de cálculo de planificación implica, habitualmente, escribir código imperativo que mezcla la lógica del problema con los detalles de la API de hojas de cálculo: letras de columna, índices de fila, cadenas de fórmula Excel y llamadas a una biblioteca de bajo nivel. El objetivo del lenguaje de dominio específico (DSL) desarrollado en este trabajo es eliminar esa mezcla, permitiendo al programador declarar *qué* debe contener el libro—tablas, columnas calculadas, reglas de coloreado condicional—sin ocuparse de *cómo* se produce el archivo.

El DSL está implementado como un conjunto de funciones y macros de Common Lisp que construyen, en tiempo de carga, un *árbol de sintaxis abstracta* (AST) cuyos nodos son instancias de clases CLOS. La canalización de compilación comprende tres etapas claramente separadas:

1. **Expansión de macros.** Cada forma del DSL (p. ej. `def-table`, `exists`, `conditional-render`) se expande en una llamada al constructor CLOS del nodo AST correspondiente. El resultado es un árbol de objetos en memoria.
2. **Generación de código.** El árbol se recorre por el backend, que traduce cada nodo a cadenas de fórmula Excel (expresiones de celda y reglas de formato condicional) y a instrucciones de construcción del libro. El backend es el único punto que conoce los nombres de columna reales, los índices de fila y las cadenas de la API de destino.
3. **Ejecución.** El programa generado se ejecuta, produciendo el archivo `.xlsx` con los datos, fórmulas y reglas de formato condicional incrustadas.

Un principio central de diseño es la *separación entre intención e implementación*: el DSL expresa la semántica del problema (“colorear los profesores que tienen un conflicto de horario”) y el backend decide el mecanismo concreto (“generar una `FormulaRule` `SUMPRODUCT` que persiste y recalcula en Excel”). El programador del DSL no escribe ninguna fórmula Excel.

1.2 Árbol de sintaxis abstracta

El árbol de sintaxis abstracta (AST) es la representación intermedia que separa la capa del DSL de la capa del generador de código. Cada concepto del problema—una tabla, una expresión de celda, un cuantificador existencial—queda capturado como un nodo AST con atributos propios. El backend opera únicamente sobre este árbol; no interpreta texto ni inspecciona las formas del DSL.

Todos los nodos del AST se definen con la macro `defclass*`, que genera simultáneamente una clase CLOS `clase-xl-nombre` y una función constructora `xl-nombre` con parámetros por clave. Por ejemplo, el nodo del cuantificador existencial se define y construye de la siguiente manera:

Listing 1.1: Patron de definicion de nodo AST con `defclass*`.

```
;; defclass* genera la clase y el constructor xl-expr-exists:
(defclass* xl-expr-exists ()
  (bind-var domain match-keys overlap-cols self-in-
    cols))

;; El constructor acepta los slots como :keywords:
(xl-expr-exists :bind-var      'r
                :domain        (xl-domain-rows :table-id nil)
                :match-keys     '(dia hora)
                :overlap-cols   '(tutor oponente presidente
                                   vocal secretario)
                :self-in-cols   nil)
```

Los nodos del AST se agrupan en cuatro categorías según su papel en el árbol.

1.2.1 Estructura del workbook

Los nodos de estructura representan los contenedores del libro de cálculo. Son los nodos de más alto nivel del árbol: el workbook agrupa hojas, las hojas agrupan regiones y las regiones agrupan tablas. Su jerarquía refleja directamente la estructura visible del archivo `.xlsx`, de modo que el programador puede razonar sobre la disposición del libro usando los mismos términos que usaría al abrirlo en Excel.

1.2.2 Expresiones de celda

Las expresiones de celda son los nodos que el backend traduce a cadenas de fórmula Excel. Abarcan tanto referencias a columnas de la misma tabla o de otra (`xl-expr-column-ref`, `xl-table-range`) como operaciones de hoja de cálculo concretas (`COUNTIF`, `COUNTA`, `SUM`) y literales de texto. Gracias a esta representación, el programador escribe (`countif rango criterio`) en lugar de componer manualmente la cadena `COUNTIF($B$4:$F$6,K4)`.

Cuadro 1.1: Nodos de estructura del workbook.

Nodo	Atributos	Papel
<code>xl-workbook</code>	<code>name</code> , <code>sheets</code>	Raíz del árbol. <code>name</code> es el nombre del archivo <code>.xlsx</code> ; <code>sheets</code> es la lista de hojas.
<code>xl-sheet</code>	<code>name</code> , <code>regions</code> , <code>fixed-expressions</code>	Una hoja del libro. Contiene regiones horizontales y expresiones en posición relativa fuera de las tablas.
<code>xl-region</code>	<code>tables</code>	Agrupar tablas colocadas una junto a otra en la misma hoja, separadas por una columna en blanco.
<code>xl-table</code>	<code>id</code> , <code>col-names</code> , <code>headers</code> , <code>contenido</code> , <code>computed</code> , <code>style-rules</code> , <code>cell-height</code> , <code>cell-width</code>	Plantilla de tabla. <code>col-names</code> enumera los símbolos DSL de columna; <code>computed</code> asocia columnas a expresiones de fórmula; <code>style-rules</code> almacena las reglas de coloreado condicional.

1.2.3 Cuantificación existencial

La cuantificación existencial es el mecanismo del DSL para expresar condiciones de la forma “existe alguna fila que cumpla ciertos criterios”. Mantenerla como un nodo AST propio, y no como una expresión booleana explícita, permite al backend generar la comprobación más adecuada según el contexto: una fórmula `SUMPRODUCT` vectorial en el caso de Excel, u otra implementación para un backend diferente.

1.2.4 Formato condicional y posicionamiento

Estos nodos permiten expresar dos capacidades que van más allá de los datos de la tabla: el coloreado condicional de celdas y la colocación de expresiones en posiciones relativas a las tablas. La separación en nodos propios garantiza que el DSL no exponga coordenadas absolutas: el backend resuelve las posiciones finales en función del tamaño real de los datos.

1.3 Definición de tablas: `def-table`

La forma central del DSL para definir datos tabulares es `def-table`. Produce una *plantilla* de tabla reutilizable: una descripción abstracta de las columnas, sus cabeceras, las fórmulas que se calculan a partir de ellas y la regla de coloreado que se aplica sobre

Cuadro 1.2: Nodos de expresion de celda.

Nodo	Atributos	Papel
xl-expr-column-ref	name, context	Referencia a la columna name en la fila actual. Produce la letra Excel de esa columna más el número de fila relativo.
xl-range	from-col, to-col	Rango de columnas contiguas de la tabla actual.
xl-table-range	table-id, from-col, to-col	Rango en una tabla específica de la región (referencia cross-table). Lo produce la forma trange .
xl-expr-countif	count-range, criteria	Formula COUNTIF sobre un rango con criterio.
xl-expr-counta	count-range	Formula COUNTA : cuenta celdas no vacías.
xl-expr-sum	count-range	Formula SUM sobre un rango.
xl-expr-string	value	Literal de texto; produce una cadena fija en la celda.
xl-expr-if	test, then, else	Condicional IF de tres ramas.
xl-expr-equals	a, b	Igualdad entre dos expresiones.
xl-expr-and, xl-expr-or	a, b	Conjunción y disyunción lógica.
xl-expr-concat	a, b	Concatenación de texto (&).

Cuadro 1.3: Nodos de cuantificacion existencial.

Nodo	Atributos	Papel
xl-expr-exists	bind-var, domain, match-keys, overlap-cols, self-in-cols	Cuantificador existencial estructurado. Declara que debe existir una fila del dominio que cumpla las condiciones expresadas por los slots semánticos. El backend decide el mecanismo de comprobación según el tipo de dominio.
xl-domain-rows	table-id	Dominio de filas. table-id = nil indica la tabla actual; un símbolo identifica una tabla específica de la región (dominio cross-table).

Cuadro 1.4: Nodos de formato condicional y posicionamiento relativo.

Nodo	Atributos	Papel
<code>xl-style-rule</code>	<code>rule-condition</code> , <code>target-columns</code>	Regla de coloreado condicional. <code>rule-condition</code> es un nodo de expresión booleana; <code>target-columns</code> enumera las columnas que se colorean cuando la condición es verdadera.
<code>xl-anchor</code>	<code>table-id</code> , <code>col-name</code> , <code>anchor-type</code>	Punto de referencia dentro de una tabla. <code>anchor-type</code> es <code>:ultima-fila</code> o <code>:primera-fila</code> .
<code>xl-nav</code>	<code>anchor</code> , <code>steps</code>	Posición calculada como un ancla más una secuencia de pasos direccionales (arriba, abajo, izquierda, derecha).
<code>xl-sheet-fixed-exprpos</code> , <code>expr</code>		Expresión colocada en una posición relativa de la hoja. <code>pos</code> es un nodo <code>xl-nav</code> ; <code>expr</code> es el nodo de expresión.

sus filas. La plantilla no contiene datos; se instancia con datos concretos al ensamblar el workbook (sección 1.7).

La sintaxis completa de `def-table` es:

Listing 1.2: Sintaxis completa de `def-table`.

```
(def-table nombre-tabla
  ((col1 "Cabecera1") (col2 "Cabecera2") ...) ; columnas
  :cell-height N          ; filas por fila logica (default 1)
  :cell-width N           ; columnas por columna logica (default
    1)
  :computed                ; formulas derivadas (opcional)
    ((colN (expresion-dsl)))
  :render                  ; regla de coloreado (opcional)
    (conditional-rendering
     :condition (expresion-booleana)
     :target-columns (col1 col2 ...)))
```

Cada par (`col` `Cabecera`) declara un nombre de columna DSL (símbolo) y el texto de su encabezado visual en la hoja de cálculo. El orden de las columnas es el orden físico en Excel; este orden es relevante cuando se usan rangos contiguos con `trange`.

La clave `:computed` asocia columnas a expresiones DSL que el compilador traduce a fórmulas Excel. Los valores iniciales de esas columnas en los datos de entrada se

ignoran: la celda siempre mostrará el resultado de la fórmula calculada.

La clave `:render` acepta una forma `conditional-rendering` que especifica bajo qué condición se colorean determinadas columnas. El mecanismo concreto lo decide el backend según el tipo de condición declarada (sección 1.6).

El listado siguiente define la tabla de profesores del caso de uso principal, incluyendo una columna calculada y una regla de coloreado condicional:

Listing 1.3: Tabla de profesores con columna computed y regla de coloreado.

```
(def-table profesores-table
  ((nombre "") (grado "") (defensas ""))
  :cell-height 1
  :cell-width 1
  :computed
    ;; Cuenta cuantas veces aparece este profesor en los cinco
    roles
    ((defensas (countif (trange defensas-table tutor
                                secretario)
                        (col nombre))))
  :render
    (conditional-rendering
     :condition
       (exists (r :rows-of defensas-table)
        :self-in (tutor oponente presidente vocal
                  secretario)
        :matching (dia hora))
     :target-columns (nombre)))
```

La variable `r` en la forma `exists` es un alias documental de la fila iterada, análogo a un alias de tabla en SQL: facilita la lectura del código sin que el generador lo evalúe directamente. La regla de coloreado declara que la celda `nombre` debe resaltarse cuando el profesor participa en alguna defensa con conflicto de horario; el mecanismo lo decide el backend (sección 1.6).

1.4 Expresiones de celda

El DSL ofrece un conjunto de formas simbólicas para construir expresiones de celda sin escribir cadenas de fórmula Excel directamente. El compilador resuelve los nombres de columna a sus letras reales, los rangos a sus coordenadas absolutas y las operaciones a su sintaxis Excel; el programador solo declara la intención.

La tabla 1.5 recoge las formas disponibles junto con su traducción.

La forma `trange` merece atención especial porque produce *referencias cross-table*: permite que una expresión en `profesores-table` cite columnas de `defensas-table` sin que el programador conozca las letras de columna reales. El compilador las resuelve en función del orden de declaración de las tablas en la región.

Cuadro 1.5: Formas de expresion del DSL y su traduccion Excel.

Forma DSL	Significado / traduccion Excel
(col nombre)	Valor de la columna nombre en la fila actual. Se resuelve a la letra Excel de esa columna más el número de fila relativo, p.ej. B4.
(trange tabla-id desde hasta)	Rango que abarca desde la columna desde hasta la columna hasta de tabla-id , para todas las filas de datos. P.ej. \$B\$4:\$F\$6.
(countif rango criterio)	Formula COUNTIF(rango, criterio).
(counta rango)	Formula COUNTA(rango): número de celdas no vacías.
(sum-range rango)	Formula SUM(rango).
(str "texto")	Literal de texto; coloca la cadena directamente en la celda.
(_equals a b)	Igualdad: produce (a = b) en la fórmula Excel.
(_and a b), (_or a b)	Conjunción y disyunción: AND(a,b), OR(a,b).
(nav ancla dir n ...)	Posición relativa: ancla más pasos direccionales. Produce una referencia de celda absoluta.

Listing 1.4: Expresiones DSL y sus formulas Excel generadas.

```
;; (col nombre) en profesores-table se resuelve a la columna K
, fila relativa
(col nombre)           ;; -> K4 (en la fila 4)

;; (trange defensas-table tutor secretario) -> rango absoluto
de los cinco roles
(trange defensas-table tutor secretario)    ;; -> $B$4:$F$6

;; La formula COUNTIF resultante
(countif (trange defensas-table tutor secretario) (col nombre)
)
;; -> COUNTIF($B$4:$F$6, K4)
```

1.5 El cuantificador existencial exists

El cuantificador **exists** permite declarar condiciones de *existencia* sobre filas de una tabla. La motivación de esta forma es que preguntas del tipo “existe otra defensa con el mismo horario” o “aparece este profesor en alguna defensa con conflicto” son muy frecuentes en problemas de planificación, pero expresarlas mediante disyunciones y bucles explícitos produce código verboso y difícil de leer.

Con `exists` el programador declara la intención semántica a alto nivel; el backend se encarga de generar la comprobación correcta para cada caso.

El cuantificador existe en dos variantes según el dominio de búsqueda.

1.5.1 Variante same-table: `:all-rows`

Esta variante busca otra fila dentro de la *misma* tabla que satisfaga condiciones de coincidencia de valores y solapamiento en columnas de rol. Se usa cuando se quiere detectar si una fila del propio conjunto tiene algún conflicto con otra fila del mismo conjunto; por ejemplo, dos defensas programadas en el mismo día y hora con al menos un integrante de tribunal en común.

Listing 1.5: Cuantificador exists sobre la tabla actual.

```
(exists (r :all-rows)
  :matching (dia hora) ; mismos valores en
    ambas filas
  :any-overlap (tutor oponente presidente vocal secretario))
  ; algun valor
    compartido en estos
    slots
```

La variable `r` es el alias de la fila del dominio que se está comprobando. La semántica de la forma anterior es: “existe otra fila de esta tabla (distinta de la actual) cuya columna `dia` y cuya columna `hora` coinciden con los de la fila actual, y que además comparte al menos un valor en las columnas de rol”.

1.5.2 Variante cross-table: `:rows-of`

Esta variante busca en las filas de *otra* tabla, relacionando el valor de la celda actual con columnas de esa tabla. Se usa cuando el elemento que se evalúa (p.ej. un profesor) y los datos de planificación (p.ej. las defensas) residen en tablas distintas.

Listing 1.6: Cuantificador exists sobre otra tabla (cross-table).

```
(exists (r :rows-of defensas-table)
  :self-in (tutor oponente presidente vocal secretario)
    ; el valor de la celda actual aparece en alguno
    de estos slots
  :matching (dia hora))
  ; ese slot tiene mas de una defensa programada
```

La semántica de la forma anterior es: “existe alguna fila de `defensas-table` en la que el valor de la celda actual (el nombre del profesor) aparece en una de las cinco columnas de rol, y cuyo par (`dia`, `hora`) tiene más de una defensa programada”. El nombre `r` identifica, en la lectura del código, la fila de `defensas-table` que se está comprobando.

En ambas variantes el alias de fila es puramente documental: el backend genera directamente una formula vectorial SUMPRODUCT y no evalúa el símbolo como variable de programa. Su papel es análogo al de un alias de tabla en una sentencia SQL SELECT.

1.6 Formato condicional declarativo

El coloreado de celdas es una herramienta habitual en las hojas de cálculo de planificación: resaltar en rojo los profesores con conflicto de horario permite detectar problemas de un vistazo. El DSL ofrece la forma `conditional-rendering` para declarar ese coloreado sin escribir fórmulas Excel ni llamadas a la API de formato.

Listing 1.7: Sintaxis de conditional-rendering.

```
(conditional-rendering
 :condition          (expresion-booleana)
 :target-columns (col1 col2 ...))
```

Cuando el nodo condición es una instancia de `xl-expr-exists`, el backend no produce colores estáticos en tiempo de compilación, sino una `FormulaRule` de Excel incrustada en el archivo `.xlsx`. Esta regla persiste en el libro y se reevalua automáticamente cada vez que el usuario edita los datos directamente en Excel, sin necesidad de volver a ejecutar el DSL.

La decisión de usar `FormulaRule` en lugar de colorear celdas directamente es exclusiva del backend: el programador del DSL no indica en ningún lugar qué mecanismo de formato se utilizará.

1.6.1 Fórmula generada para la variante same-table

Cuando `exists` usa `:all-rows`, el backend genera una formula SUMPRODUCT que, para cada fila, multiplica tres factores: coincidencia en las columnas de agrupación (`:matching`), exclusión de la fila actual (para no comparar una fila consigo misma) y solapamiento en al menos una columna de rol (`:any-overlap`).

Para una regla con `:matching` (dia hora) y `:any-overlap` (tutor ...secretario), el backend genera la siguiente formula, aplicada al rango de la tabla:

Listing 1.8: Formula Excel CF para la variante same-table (escrita para la fila 4).

```
=SUMPRODUCT(
  ($G$4:$G$6=$G4) *           -- dia coincide
  ($H$4:$H$6=$H4) *           -- hora coincide
  (ROW($B$4:$B$6)<>ROW($B4)) * -- excluir la fila actual
  (($B$4:$B$6=$B4) + ($C$4:$C$6=$C4) + ... + ($F$4:$F$6=$F4)
) > 0
```

Las referencias con columna fija y fila relativa (p.ej. `$G4`) hacen que la fórmula se evalúe correctamente para cada fila del rango: al desplazarse de la fila 4 a la fila 5, las referencias de “fila actual” avanzan automáticamente con la celda.

1.6.2 Fórmula generada para la variante cross-table

Cuando `exists` usa `:rows-of`, el backend genera una formula `SUMPRODUCT` que combina dos condiciones: que el nombre del profesor aparezca en al menos una columna de rol de la tabla de defensas (`:self-in`), y que ese slot de horario tenga más de una defensa programada (`:matching` con `COUNTIFS`).

Para la regla de `profesores-table` con `:self-in` (`tutor ...secretario`) y `:matching` (`dia hora`), el backend genera la siguiente formula, aplicada al rango de la columna `nombre`:

Listing 1.9: Formula Excel CF para la variante cross-table (escrita para K4).

```
=SUMPRODUCT (
  (( $B$4 : $B$6 = $K4 ) + ( $C$4 : $C$6 = $K4 ) + ( $D$4 : $D$6 = $K4 ) +
    ( $E$4 : $E$6 = $K4 ) + ( $F$4 : $F$6 = $K4 ) ) *      -- aparece en
    algun rol
  ( COUNTIFS ( $G$4 : $G$6 , $G$4 : $G$6 ,
              $H$4 : $H$6 , $H$4 : $H$6 ) > 1 )          -- slot con mas
              de 1 defensa
) > 0
```

La referencia `$K4` corresponde al nombre del profesor en la celda actual (columna K bloqueada, fila relativa). Los rangos `$B$4:$B$6` a `$F$4:$F$6` son las columnas de rol de `defensas-table`, completamente absolutas. `COUNTIFS` cuenta cuántas filas comparten el mismo `dia` y `hora`; si el resultado es mayor que 1, ese slot tiene conflicto.

El resultado es que los profesores marcados en rojo son exactamente aquellos que aparecen en al menos una defensa cuyo slot horario tiene más de una defensa programada.

1.7 Ensamblaje del workbook

Una vez definidas las plantillas de tabla, el programador ensambla el workbook con tres macros: `libro` crea el nodo raíz del AST con el nombre del archivo de salida; `hoja` agrupa tablas y expresiones fijas en una pestaña del libro; `tabla` instancia una plantilla (`def-table`) con datos concretos.

Listing 1.10: Ensamblaje del workbook con libro, hoja y tabla.

```
(libro nombre-libro
 :filename "Archivo.xlsx"
 :hojas (list
  (hoja "NombreHoja"
   (tabla plantilla1 :data *DATOS-1*)
   (tabla plantilla2 :data *DATOS-2*)
```

```
:fixed-expressions
  (((nav ...) (expresion))
   ((nav ...) (expresion))))))
```

Cuando una hoja recibe más de una tabla, el generador las coloca en la misma *región horizontal*: las tablas se disponen de izquierda a derecha separadas por una columna en blanco. Este diseño permite que las referencias cross-table (**trange**) se resuelvan dentro de la misma región con letras de columna absolutas correctas.

La clave `:fixed-expressions` acepta una lista de pares (**posición**, **expresión**). La posición se especifica con **nav** (sección 1.8); la expresión puede ser cualquier forma del DSL. Estas expresiones quedan fuera de las tablas y su posición se recalcula según el tamaño real de los datos, de modo que los totales de pie de tabla se desplazan automáticamente si se añaden más filas.

1.8 Posicionamiento relativo: nav

Las expresiones fijas de una hoja (etiquetas de totales, celdas de resumen) no pueden tener coordenadas absolutas codificadas, pues esas coordenadas dependen de cuántas filas de datos tenga cada tabla en cada ejecución. La forma **nav** resuelve esto: calcula la posición de una celda como un *ancla* fija en una tabla más una secuencia de pasos direccionales.

Las anclas disponibles son **ultima-fila** y **primera-fila**, que identifican la última o primera fila de datos de una columna de una tabla. A partir del ancla se pueden encadenar pasos **abajo**, **arriba**, **izquierda** y **derecha** con su cantidad:

Listing 1.11: Sintaxis de nav y anclas de tabla.

```
;; Una fila debajo de la ultima fila de datos
(nav (ultima-fila tabla-id col-name) abajo 1)

;; Una fila debajo y dos columnas a la derecha
(nav (ultima-fila tabla-id col-name) abajo 1 derecha 2)

;; Una fila encima de la primera fila de datos
(nav (primera-fila tabla-id col-name) arriba 1)
```

El compilador resuelve **ultima-fila** a la fila Excel correspondiente en tiempo de generación y produce una referencia de celda absoluta que siempre apunta al lugar correcto, independientemente del número de filas de datos.

Listing 1.12: Expresiones fijas de totales con posicionamiento relativo.

```
;; Etiqueta una fila debajo de la ultima defensa
((nav (ultima-fila defensas-table estudiante) abajo 1)
 (str "Total defensas:"))
;; Valor COUNTA en la columna siguiente
((nav (ultima-fila defensas-table estudiante) abajo 1 derecha
 1)
```

```
(counta (trange defensas-table estudiante)))

;; Suma de defensas por profesor
((nav (ultima-fila profesores-table nombre) abajo 1 derecha 2)
 (sum-range (trange profesores-table defensas)))
```

1.9 Generación y ejecución

Las dos últimas formas del DSL cierran el ciclo de compilación: `xl-generate` recorre el árbol AST y produce el programa de construcción del libro; `xl-run-generated` ejecuta ese programa y crea el archivo `.xlsx`.

Listing 1.13: Compilacion del AST y ejecucion.

```
;; Recorre el AST y genera el programa de construccion del
  libro
(xl-generate horario-tesis "horario-tesis.py")

;; Ejecuta el programa generado y crea el archivo .xlsx
(xl-run-generated "horario-tesis.py")
```

El archivo `.xlsx` resultante contiene tanto los valores y fórmulas de celda como las `FormulaRule` de formato condicional. Estas últimas permanecen activas cuando el usuario abre y edita el libro directamente en Excel: el coloreado se recalcula sin necesidad de volver a ejecutar el DSL.

1.10 Ejemplo completo: Horario de Defensas de Tesis

El caso de uso principal del DSL es la generación automática de un horario de defensas de tesis con dos tablas relacionadas. La primera tabla, `defensas-table`, registra cada defensa con su tribunal (cinco roles: tutor, oponente, presidente, vocal y secretario), su slot horario (día y hora) y el local. La segunda tabla, `profesores-table`, enumera los profesores con su grado académico y el número de defensas en que participan; además, sus filas se colorean en rojo cuando el profesor tiene un conflicto de horario, es decir, cuando aparece en dos defensas programadas en el mismo slot.

El listado siguiente es el programa DSL completo. En él se pueden observar todas las construcciones descritas en el capítulo: definición de plantillas de tabla, `columna computed`, cuantificador `exists` cross-table, formato condicional declarativo, ensamblaje del workbook y posicionamiento relativo con `nav`.

Listing 1.14: Programa DSL completo: Horario de Defensas de Tesis.

```
(load "dsl-directo.lisp")
(load "data-tutorial.lisp") ; *DATOS-DEFENSAS*, *DATOS-
  PROFESORES*
```

```

;; -- TABLA 1: defensas
-----
;; Nueve columnas: cinco de rol (tutor...secretario) mas dia,
hora, local.
;; Las cinco columnas de rol son contiguas para poder
referenciarlas como
;; rango con (trange defensas-table tutor secretario).
;; No tiene regla de coloreado; el analisis se delega a
profesores-table.
(def-table defensas-table
  ((estudiante "") (tutor "") (oponente "") (presidente "")
   (vocal "") (secretario "") (dia "") (hora "") (local ""))
  :cell-height 1
  :cell-width 1)

;; -- TABLA 2: profesores
-----
;; La columna "defensas" cuenta apariciones del nombre en los
cinco roles.
;; El formato condicional marca en rojo a quienes tienen
conflicto.
(def-table profesores-table
  ((nombre "") (grado "") (defensas ""))
  :cell-height 1
  :cell-width 1
  :computed
  ((defensas (countif (trange defensas-table tutor
                              secretario)
                      (col nombre))))
  :render
  (conditional-rendering
   :condition
   (exists (r :rows-of defensas-table)
    :self-in (tutor oponente presidente vocal
               secretario)
    :matching (dia hora))
   :target-columns (nombre)))

;; -- WORKBOOK
-----
(libro horario-tesis
 :filename "Horario_Tesis.xlsx"
 :hojas (list
  (hoja "Defensas"
   (tabla defensas-table :data *DATOS-DEFENSAS*)
   (tabla profesores-table :data *DATOS-PROFESORES*)
   :fixed-expressions
   (((nav (ultima-fila defensas-table estudiante) abajo
            1)
    (str "Total defensas:"))))

```

```

((nav (ultima-fila defensas-table estudiante) abajo 1
  derecha 1)
 (counta (trange defensas-table estudiante)))
((nav (ultima-fila profesores-table nombre) abajo 1)
 (str "Total profesores:"))
((nav (ultima-fila profesores-table nombre) abajo 1
  derecha 2)
 (sum-range (trange profesores-table defensas))))))

(xl-generate horario-tesis "horario-tesis.py")
(xl-run-generated "horario-tesis.py")

```

Los datos de entrada contienen tres defensas y seis profesores. Dos de las defensas comparten el slot Lunes 10:00 con roles solapados. El programa genera `Horario_Tesis.xlsx` con la estructura siguiente:

- Columnas A–I: `defensas-table` con tres filas de datos.
- Columna J: separador en blanco de la región.
- Columnas K–M: `profesores-table` con seis filas de datos y la columna `Defensas` calculada mediante `COUNTIF`.
- Fila de totales debajo de cada tabla: conteo de defensas con `COUNTA` y suma de defensas por profesor con `SUM`.
- Formato condicional `FormulaRule` aplicado al rango de la columna `nombre`: los profesores que participan en al menos una defensa con conflicto de horario aparecen resaltados en rojo.

La regla de coloreado generada implementa exactamente la semántica del `exists` declarado en el DSL:

Listing 1.15: `FormulaRule SUMPRODUCT+COUNTIFS` incrustada en el `.xlsx`.

```

-- Rango: K4:K9    Formula escrita para K4 (fila relativa)
=SUMPRODUCT(
  (($B$4:$B$6=$K4) + ($C$4:$C$6=$K4) + ($D$4:$D$6=$K4) +
   ($E$4:$E$6=$K4) + ($F$4:$F$6=$K4)) *
  (COUNTIFS($G$4:$G$6,$G$4:$G$6,
             $H$4:$H$6,$H$4:$H$6) > 1)
) > 0

```

La primera parte comprueba si el nombre del profesor (celda K4) aparece en alguna de las cinco columnas de rol de `defensas-table`; la segunda parte comprueba si ese slot tiene más de una defensa programada. Al ser una `FormulaRule` incrustada en el archivo Excel, cualquier edición posterior de los datos provoca una reevaluación automática del coloreado, sin necesidad de volver a ejecutar el DSL.